



ICPP 2026

My Submissions

Make a New Submission

My Conflicts

All Reviews of pap385 [details](#)

Decision: Reject
Submission: GenoX: Exploiting Thread Signals for Dynamic CPU Scaling in Genomics Workflows
Contributors: Grossman, Gunawi, Kim, Putra

Key for the below column headings [show](#)

Summary of Reviews of pap385s1									
Reviewer	rlv ⓘ	snd ⓘ	imp ⓘ	orig ⓘ	pres ⓘ	recom ⓘ	conf ⓘ	bp ⓘ	
Reviewer 1	4	3	3	2	2	Weak REJECT (2)	3	0	
Reviewer 2	4	3	3	3	3	Weak REJECT (2)	3	0	
Reviewer 3	5	5	4	5	4	Weak ACCEPT (3)	4	0	
Averages:	4.3	3.7	3.3	3.3	3.0		2.3	3.3	0.0

- Sections Below
- Full Reviews · [jump down](#)
 - Committee Discussion with Authors · [jump down](#)
 - Committee Comments · [jump down](#)

Full Reviews

Reviewer 1

[↑ top](#)

► Paper summary

monospace

This paper proposes GenoX, a quota scheduler for genomic workflows, which consist of complex workflows where steps are executed in separate containers with changing thread-counts even within a single container and between containers. In the system, a monitor component monitors the number of threads used inside each container and adjusts the quota of the container if that count changes. The evaluation compares this approach against utilization-based approaches that adjust

the quota based on utilization metrics and shows that the faster adjustment based on active threads yields benefits.

► Paper strengths

markdown monospace

- This appears to yield an improvement over existing utilization-based approaches
- The paper is well-written and structured
- Relevant for a significant community

► Paper weaknesses

markdown monospace

- Unclear motivation
- Lack of evaluation against unconstrained execution.
- Some design choices are not clearly explained.

► Detailed comments

monospace

The motivation for this work is not clear. The paper does not properly introduce the architecture of these workflows and the associated challenges. Section 3.2 mentions that genomic workflows are executed in batches, which is not clearly mentioned before. The underlying challenge appears to be that multiple workflows spawn multiple parallel jobs on the same node, each utilizing varying numbers of threads.

It is not clear why the quota mechanism is needed at all. If the system dynamically raises the quota when new threads become active (Section 4.2), what is the benefit of a quota in the first place? Why do these workflows not get access to all cores and have the Linux scheduler take care of quotas? The evaluation should compare against a non-quota workflow execution.

Some design choices are not clearly explained. Why use a 5ms polling interval and not 3 or 8? Was there any evaluation as to the trade-off in choosing different intervals?

The term "signal" is ambiguous in this context as it is an operating system concept that applies to threads. I would suggest reconsidering its use, including in the title.

The discussion in Section 5.4 is not clear. The figures (13, 14) show a gantt chart of workflow executions but do not seem to show the difference in execution between different approaches. It is not clear how the impacts of other approaches are derived from these figures.

Scored Review Questions

	rlv ⓘ	snd ⓘ	imp ⓘ	orig ⓘ	pres ⓘ	recom ⓘ	conf ⓘ	bp ⓘ
pap385s1	4	3	3	2	2	Weak REJECT (2)	3	0

Reviewer 2

[↑ top](#)

► Paper summary

monospace

The paper proposes GenoX, a dynamic CPU quota management system for containerized genomics workflows. It argues that CPU utilization is unreliable for DNA-Seq and RNA-Seq workflows because of input-dependent performance variability, contention-induced slowdown, dynamic workflow branching, and millisecond-scale idle bursts. GenoX uses per-container thread counts, monitored every 5 ms, to adjust CPU quotas through Linux cgroups. The paper evaluates GenoX against utilization-based baselines on representative genomics applications and complete DNA-Seq/RNA-Seq workflows.

► Paper strengths

markdown monospace

1. The paper addresses a real problem in genomics workflows, where applications run in shared clusters and compete for CPU resources.
2. The measurement study is useful and shows concrete behaviors such as input-dependent performance variability, inflatable parallelism, stall heterogeneity, and millisecond-scale idle bursts.
3. GenoX is practical because it uses standard Linux cgroups and does not require changes to workflow engines or bioinformatics applications.
4. The evaluation includes both application-level case studies and workflow-level experiments, and the reported runtime improvements are meaningful in the tested setting.

► Paper weaknesses

markdown monospace

1. The novelty is somewhat limited. The paper's stronger contribution is in showing that thread count is a useful runtime signal for genomics workflows, rather than in proposing a fundamentally new scheduling technique.
2. The paper gives an intuitive argument for using thread count as a signal, but it does not sufficiently characterize when thread count is a reliable proxy for useful CPU demand, especially for blocked, sleeping, or I/O-heavy threads.
3. The 1:1 mapping between active threads and CPU quota is not analyzed deeply enough, even though different threads may have different CPU needs.
4. The evaluation is somewhat narrow, since it mainly uses DNA-Seq/RNA-Seq workflows and does not test many co-located workflows under heavier contention.

► Detailed comments

markdown monospace

1. The paper should explain more clearly the assumptions under which thread count is a better signal than utilization, and where this assumption may fail.
2. The authors should evaluate more contention scenarios, especially with many workflows running at the same time on the same node.
3. The paper should include stronger comparisons, such as well-tuned static CPU allocation, workflow-aware scheduling, or policies that combine thread count with utilization or stall

- information.
- The evaluation should report repeated runs, variance/error bars, and the overhead of monitoring thread counts every 5 ms.
 - The work is best understood as a practical, domain-specific resource-management system for genomics workflows rather than a broadly new CPU scheduling technique.

Scored Review Questions

	rlv ⓘ	snd ⓘ	imp ⓘ	orig ⓘ	pres ⓘ	recom ⓘ	conf ⓘ	bp ⓘ
pap385s1	4	3	3	3	3	Weak REJECT (2)	3	0

Reviewer 3

↑ top

▸ Paper summary

monospace

This paper proposes a thread count based dynamic CPU scaling approach called GenoX and its need for Genomic workflows.

▸ Paper strengths

monospace

This paper presents a measurement-based problem characterization, along with the primary issues faced in executing Genomic Workflows, which is well explained in the initial sections, and the design based on polling thread count and dynamic assignment of CPU quota through c-groups, along with thorough characterization and evaluation from micro-benchmarks to real-world workflows, is appreciated.

▸ Paper weaknesses

markdown monospace

- The proposed model is a CPU-only allocator, and the paper does not characterize or discuss the memory pressure impact of these workflows. In microbenchmarks evaluation it shows the thread count variations are in 10's, but with real DNA, RNA pipelines with production datasets what ranges the thread count variations occur and the impact of memory pressure using this allocator need a discussion, as today most of the server architectures are disaggregated where the impact of allocating more CPUs to a specific application would definitely create memory latency effects.
- The paper explains that managing dynamic CPU allocation reduces contention and reduces stalls, Try to add a discussion how this would address I/O based contention.

▸ Detailed comments

Please refer to above section

Scored Review Questions

	rlv ⓘ	snd ⓘ	imp ⓘ	orig ⓘ	pres ⓘ	recom ⓘ	conf ⓘ	bp ⓘ
pap385s1	5	5	4	5	4	Weak ACCEPT (3)	4	0

Committee Discussion with Authors

[↑ top](#)

No discussion is available at this time.

Committee Comments

[↑ top](#)

Committee Comments for Authors

None